

NATIONAL ROBOTICS WEEKEND 8.0 · INSAT · DÉFI SOPAL & SOPALTEC

Smart Core Warehouse

A complete, beginner-friendly walkthrough of what was built, how it works,
and why every piece exists.

Backend + Database

Live Dashboard

ESP32 Simulation

Automatic Decisions

Written so that no prior knowledge of the code is needed to follow it.
Contract version 1.7 · Prepared for the project owner

Contents

1. The problem, in plain words
2. The big picture — three layers
3. Words you'll need (glossary)
4. The life of one box, step by step
5. How the system "sees" what's in a box — identification
6. How the system counts what's in a box — quantity
7. Tracking the 24-hour drying rule
8. Automatically choosing the right box (FIFO)
9. When something goes wrong — quarantine and recovery
10. The dashboard — what you see and click
11. The ESP32 — simulating the real hardware
12. How we know it actually works (testing)
13. The file map — what lives where
14. Scoring — how this maps to the official brief
15. Where things stand right now

1. The problem, in plain words

SOPAL & SOPALTEC is a foundry — a factory that casts metal parts. Before molten metal is poured, it needs a mould, and part of that mould is made of small sand shapes called **cores** (in French, *noyaux*). Cores are made, then they need to sit and harden ("cure") for at least 24 hours before they can be used. In the meantime they sit in boxes, in a warehouse, waiting.

The competition brief (the CDC — *Cahier des Charges*, i.e. the spec sheet) asks teams to build a "smart" version of that warehouse: something that can automatically answer, at any moment, six questions:

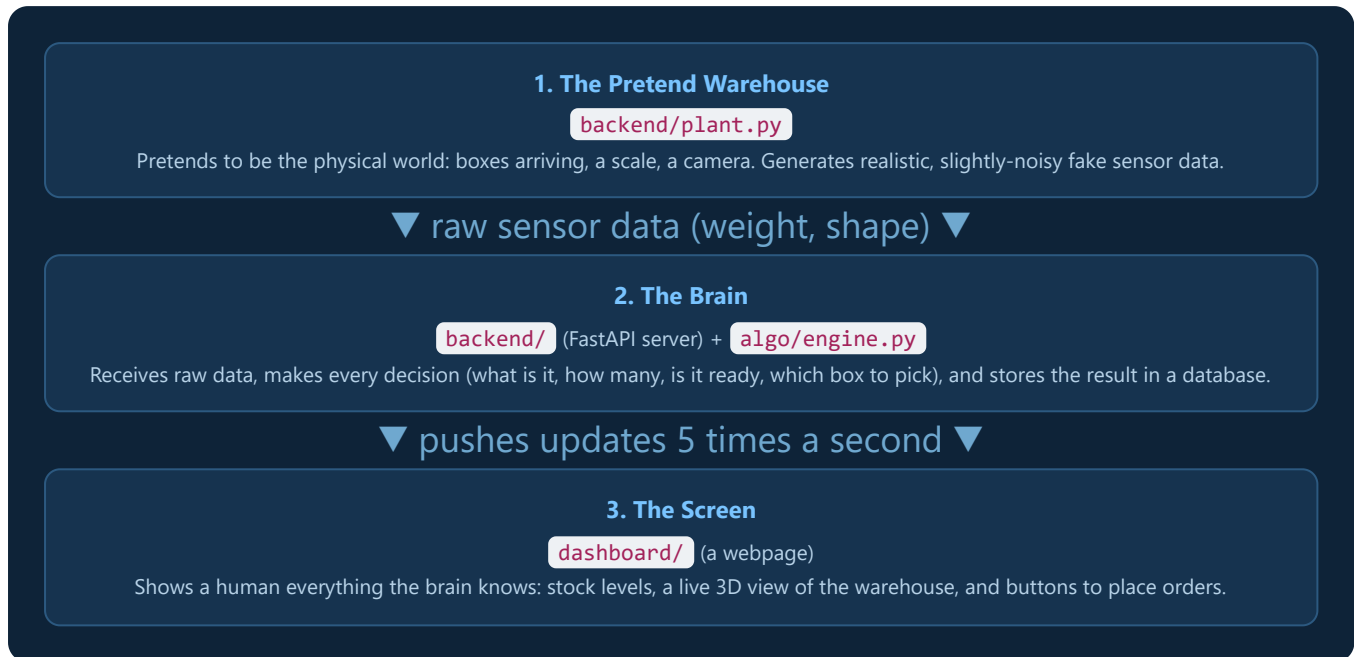
Which core type is this? How many are in the box? Which box is it in? How long has it been sitting? Is it ready yet? And which box should we grab first?

This repository ([scw/](#) , short for Smart Core Warehouse) is the software answer to that brief: a simulated warehouse that receives boxes, figures out what's in them without a human checking, tracks their drying time, and automatically tells the operator which box to send out when an order comes in.

 **Plain-English analogy:** *Think of it like a smart pantry that scans every grocery bag as it comes in, weighs it, double-checks the weight against a photo of what's inside, writes down the exact minute it arrived, and — when you say "I need 3 tomatoes" — tells you exactly which bag to open first, without you ever having to remember what's where or how long anything has been sitting.*

2. The big picture — three layers

Everything in this project is one of three things:



A fourth, optional piece sits alongside the pretend warehouse: a real (or simulated) **ESP32 microcontroller** — a small, cheap computer chip used in electronics projects — that stands in for an actual physical scale on the factory floor. It talks to the Brain over the internet using a messaging protocol called **MQTT** (think of it as a very lightweight, one-way radio channel: the chip "broadcasts" numbers, and the Brain "listens").

Why split it this way? Because it means the exact same Brain code runs whether the sensor is a real chip on a table at the competition, a chip simulated online (Wokwi), or just a fallback random-number generator inside the pretend warehouse. If the real hardware misbehaves during the demo, the system quietly keeps working — the jury never sees a crash.

3. Words you'll need

Core (noyau)

The small sand part being tracked. Comes in different reference types (models), each with its own size and weight.

Barcode

A code printed on each box's label saying "this box claims to be reference NY-114". It's a claim, not proof — the system double-checks it (see section 5).

Backend

The program that does all the thinking. Never seen directly by a user — it just answers questions over the network.

Database (SQLite)

A single file (`scw.db`) that stores every box, every order, every event that ever happened, like a very organized spreadsheet the Backend reads and writes.

API / REST

The set of "questions you're allowed to ask the Backend" — e.g. "give me the current stock", "place this order". Each one is a small web address (URL) the dashboard calls.

WebSocket

A live, always-open phone line between the dashboard and the Backend, used to push updates to the screen instantly (5 times a second) instead of the screen having to keep asking "anything new? anything new?".

MQTT

The messaging channel used to talk to the ESP32 chip. The chip publishes raw numbers; the Backend subscribes and listens.

Simulated time (t_sim)

The entire warehouse runs on a fake clock that can be fast-forwarded. This is critical: the brief needs a box to sit for 24 real hours before it's "ready", but nobody can wait 24 hours during a 10-minute demo. So the system has its own clock that can jump forward on command ("jump 26 simulated hours"), and nothing anywhere reads the real-world clock.

FIFO

"First In, First Out" — always use the oldest available box of a given reference before a newer one. Same principle as rotating stock in a fridge: oldest milk carton goes to the front.

Confidence (HAUTE / MOYENNE / NULLE)

French for HIGH / MEDIUM / NONE. Every automatic decision (what's in the box, how many) comes with one of these three labels, so the system never silently pretends to be sure when it isn't.

Quarantine

A holding state for a box the system couldn't confidently make sense of. It's set aside instead of being guessed into the wrong shelf.

ESP32

A small, inexpensive Wi-Fi-capable microcontroller (a tiny computer chip), commonly used to read sensors and talk to servers. Here it stands in for the physical scale on the factory floor.

Wokwi

A free website that simulates electronics circuits — including an ESP32 — in the browser, so the firmware can be demoed without owning the physical chip.

4. The life of one box, step by step

Here is exactly what happens, in order, from the moment a box of cores shows up to the moment it leaves the warehouse.

- 1 A box arrives.** A barcode is scanned (claiming a reference, e.g. "NY-114"). The box goes onto a scale, and — in this project's design — is also "seen" by a simulated camera/vision station that independently reads the box's shape and roughly counts what's visible inside.
- 2 Two independent checks run at once.** *Identification:* does the barcode's claimed reference match what the vision station's shape reading says it should be? *Quantity:* does $\text{weight} \div (\text{that reference's known per-core mass})$ roughly match what the vision station counted by eye? Section 5 and 6 explain exactly how these are scored.
- 3 A verdict is reached.** If everything lines up, the box is accepted and stamped with the exact simulated-time second it arrived. If the two checks disagree too much, the box is instead marked **QUARANTINE** and set aside (section 9) rather than filed under a guess.
- 4 The box starts curing.** An accepted box is assigned a shelf slot and enters the **DRYING** state. A 24-simulated-hour countdown starts from its arrival stamp.
- 5 24 hours pass (simulated).** The system automatically flips the box to **READY** the instant its timer completes — nobody has to check on it, and the dashboard flashes a banner announcing it.
- 6 An order comes in.** Someone (an operator, or the demo) asks for, say, "30 units of NY-114". The system looks at every READY box of that reference, oldest first, and works out exactly which boxes — and how much of each — will cover the order (section 8).
- 7 The order is confirmed.** The chosen quantity is deducted from each box. A box that's fully used empties and frees its shelf slot; a box that's only partly used keeps its remaining stock, in the same slot, with its *original* arrival time (so it stays first in line next time — no fairness is lost by only taking part of it).
- 8 If there isn't enough stock at all,** the system doesn't just say "no" — it opens a tracked "production batch" reserved for that specific order, and automatically ships it, complete, the moment the last box in it finishes curing (section 8).

 **Plain-English analogy:** *This is the exact same lifecycle as a delivery-app order: item arrives at the warehouse → gets scanned and checked → shelved → waits until it's "in date"/ready → picked when someone orders it → partially or fully removed from the shelf → shelf freed if empty.*

5. How the system "sees" what's in a box — identification

Criterion 1 of the brief (worth 15 of the 160 points) asks: can the system tell, on its own, what type of core is in a box? A barcode alone answers that — but a barcode is just a sticker; anyone could put the wrong sticker on the wrong box, by accident or otherwise. A jury is entitled to ask "what if the label is wrong?" — so the design deliberately does not trust the barcode blindly.

Instead, a simulated **vision station** independently measures the box's physical shape (length, width, height, and how many holes the core pattern has) and compares that measurement against the known shape of every reference in the catalogue, picking whichever one is the closest match. This all happens in one small, dependency-free function,

```
algo/engine.py :: identify_core .
```

The two readings — barcode and vision — are then cross-checked:

HAUTE

Barcode and vision agree on the reference. Full confidence.

MOYENNE

Vision is a little uncertain about the shape, but doesn't contradict the barcode. Accepted, flagged as medium confidence.

NULLE → Quarantine

Vision confidently sees a *different* reference than the barcode claims. The box is set aside rather than trusted.

Why this matters for the demo. During the live presentation, one of the "wow" moments is deliberately triggering a mismatch (sticking the wrong label on a box, in the simulation) and showing the jury that the system catches it every single time — this was actually tested against every quantity from 1 to 80 cores, across all four reference types, with a 100% catch rate, so this claim can be made on stage with confidence.

6. How the system counts what's in a box — quantity

Criterion 2 (15 points) asks for an automatic core count. The obvious approach — weigh the box, subtract the empty box's weight, divide by one core's weight — works, but naive versions of it are fragile: real cores are never perfectly identical in weight, a scale has its own small error, and a system that isn't built to expect that noise will mis-count boxes far more often than a jury would accept.

This system uses the same two-sensor idea as identification:

Signal	What it measures	Where
Weight	Total box weight ÷ that reference's known per-core mass ≈ a count	<code>algo/engine.py :: assess_box</code>
Vision	The simulated camera's own direct count of visible cores	<code>backend/plant.py :: simulate_vision</code>

The two counts are then reconciled:

- **Agree exactly** → **HAUTE** confidence, count accepted.
- **Off by one or two** → **MOYENNE**, accepted at the lower (safer) figure — small camera occlusion is normal and shouldn't block a good box.
- **Off by three or more**, or the weight math simply doesn't land near any whole number → **QUARANTINE**. Something is genuinely wrong; don't guess.

 **Plain-English analogy:** *Like a cashier who weighs a bag of apples and glances in the bag: if the scale says "about 8 apples" and a glance confirms "yep, 8", no problem. If the scale says 8 but there are clearly only 2 in the bag, something's off and it gets a second look instead of being rung up.*

Verified with numbers, not just logic. Realistic per-core weight variation (about 3%, which is normal for cast sand parts) was simulated across 800 test boxes, and the false-quarantine rate — good boxes wrongly getting flagged — came out to 0.23%, comfortably under a 1% target that was set as the bar for "trustworthy in front of a jury".

7. Tracking the 24-hour drying rule

Criterion 4 (10 points) is the simplest rule in the whole system, deliberately: **every core, of every reference, in every simulated weather condition, must cure for exactly 24 hours** before it can be shipped. There is no adaptive model that tries to guess a shorter or longer cure time from temperature or humidity — that idea was considered and dropped early on, because the brief never actually asks for it, and a simpler, provably-correct rule is safer to defend under jury questioning than a more "clever" one that's harder to explain or verify live.

Temperature and humidity are still recorded for every arriving box and shown on the dashboard — they're just evidence, not an input to any calculation. Behind the scenes, a background loop checks, several times a second, whether any **DRYING** box has crossed its 24-hour mark, and the moment one does, it's automatically flipped to **READY** and a banner appears on screen — nobody has to click anything for a box to "become available".

8. Automatically choosing the right box (FIFO)

Criteria 5 and 6 (15 + 15 points) ask the system to classify stock by type/quantity/location/age, and to automatically propose the correct box for a request. When an order like "30 units of NY-114" comes in, the engine:

- 1 **Finds every READY box** of that reference, sorted strictly oldest-arrival-first. Boxes already reserved for a specific production order are excluded — they're spoken for and can't be double-booked by someone else's order.
- 2 **Takes only what's needed from each box in turn** — this is a deliberate design choice called a *partial pick*: if the oldest box has 22 and the order needs 30, it takes all 22 from that box and only the remaining 8 from the next-oldest box, rather than being forced to take that whole second box (which would have wastefully reserved 40 for a 30-unit order).
- 3 **If existing stock genuinely can't cover the order**, instead of a flat refusal, the system opens a tracked "production batch" reserved specifically for that order, and estimates when the shortfall will be covered by boxes already curing in the pipeline.
- 4 **The moment every box in that batch finishes curing**, the whole batch ships automatically, together — no manual button press required, though one exists as a backup.

Why partial picks matter for the demo. A jury member can ask for an odd, awkward number (say "17"), and the system will show it landing on exactly 17 across two boxes, rather than visibly over-allocating to 40 out of what looks like laziness. This is the kind of detail that separates "it technically works" from "it clearly works well".

9. When something goes wrong — quarantine and recovery

Any box the system can't confidently make sense of — a barcode/vision mismatch, an unreadable label, a weight/vision count gap too large to trust — goes to **QUARANTINE** instead of being silently forced into a guess. Two things can happen to it next:

Action	What it does	When to use it
Recount	The box is re-weighed and re-scanned, and re-evaluated exactly like a fresh arrival. If it now checks out, it re-enters the drying cycle from that moment.	The original problem was a fluke or human error that's since been fixed (wrong label replaced, box re-weighed properly).
Archive	The box is permanently closed out of the active warehouse.	The box is a genuine dead end (e.g. a barcode that was never registered to any known reference at all).

Both actions are available as buttons directly on the dashboard next to each quarantined box.

10. The dashboard — what you see and click

The dashboard is a single webpage (open `http://localhost:8000` once the server is running) split into a few panels, all updating live, five times a second:

- **KPIs** at the top — headline numbers (total boxes, total stock, orders pending).
- **Stock by reference** — how much of each core type is on hand, ready vs still drying.
- **Proposal panel** — where a demand/order is typed in, and the system shows exactly which boxes it would pick, including any partial pick or "not enough stock yet, ready in approximately X hours" answer.
- **Inventory table** — every box, its reference, quantity, shelf slot, age, and state, with a small tooltip showing the vision station's own reading for extra transparency.
- **Quarantine panel** — any box needing attention, with Recount/Archive buttons.
- **Event log** — a running, human-readable history of everything that's happened (arrivals, cures completing, shipments, quarantines) — this doubles as an audit trail.
- **3D digital twin** — a live, procedurally-drawn 3D view of the actual rack and crane, built with Three.js, that moves crates around as the real state changes. (This is a cosmetic visual for the dashboard criterion — the separate, formal mechanical/CAD design is its own deliverable outside this repository.)

Handy keyboard shortcuts used during the demo (all defined in `dashboard/app.js`): **S** load a demo scenario, **A** simulate a box arrival, **D** place a demand/order, **C** confirm it, **J** jump the simulated clock forward.

Language is switchable between French and English from a single line in `dashboard/labels.js` — every piece of on-screen text lives in that one file, so nothing has to be hunted down across the codebase before a French-speaking jury.

11. The ESP32 — simulating the real hardware

Criterion 9 (15 points) asks for a working embedded-systems simulation. This project uses **Wokwi**, a free browser-based electronics simulator, to run the exact same firmware code (`firmware/sketch.ino`) that would run on a real ESP32 chip.

The simulated circuit includes:

- A potentiometer (a knob) standing in for a real load-cell scale, since Wokwi has no native load-cell part — turning the knob simulates a box being placed on a scale.
- A DHT22 sensor for temperature/humidity.
- A small OLED screen physically showing the barcode, live weight, and the verdict, exactly as an operator standing at the physical station would see it.
- A push button to manually mark "box placement done".

An important design principle: the chip is never told the answer. It only ever reads and reports a raw measurement (a voltage reading standing in for weight). It has no idea what reference it's looking at, or how many cores that should be — *all* of that reasoning happens up in the Backend (section 5–6), using the barcode and vision data the chip knows nothing about. This separation is exactly what makes the embedded-simulation criterion defensible if a juror asks "wait, is the chip actually doing anything, or is it just a fancy button?" — yes, it's doing real, independent sensor work; the intelligence simply lives where it should, in the software layer, not baked into inflexible chip firmware.

As a safety net, `tools/fake_device.py` is a plain Python script that sends byte-for-byte identical messages to the real firmware, so the whole system can be demoed even if Wokwi itself has a bad moment on stage.

12. How we know it actually works (testing)

Rather than "it seemed to work when I clicked around", every rule described above is backed by an automated test that fails loudly if the rule is ever broken by a future change. There are five layers, each catching a different kind of mistake:

Test file	What it checks	Needs a running server?
<code>algo/test_engine.py</code>	43 tests of pure decision logic (identification, counting, FIFO, quarantine) in isolation — the fastest and most detailed layer.	No
<code>tools/test_backend.py</code>	Database-level guarantees: e.g. an order can't be double-confirmed, a batch box can't be stolen by another order.	No (uses a throwaway database file)
<code>tools/smoke.py</code>	A full, real, end-to-end walk through the exact path a jury will watch: boxes arrive, cure, get picked, ship.	Yes
<code>tools/mqtt_probe.py</code>	Checks the messaging layer to the ESP32 handles duplicate or malformed messages safely.	Yes
<code>tools/test_firmware_contract.py</code>	Makes sure the ESP32 firmware and the Backend's expectations of it never silently drift apart.	No

At the time of writing, all of these pass cleanly ("ALL GREEN"), and a separate, always-on 29-point consistency checker (`backend/consistency.py` , viewable live at </api/db/check>) continuously re-verifies that the database itself never contains a contradiction (e.g. a box marked both empty and reserved).

13. The file map — what lives where

```
scw/
├─ run.sh           one command starts the whole system
├─ algo/
│   └─ engine.py    EVERY decision the system makes, in one place.
│                   Deliberately has zero network/database code, so
│                   it can be tested in under a second and reasoned
│                   about on a whiteboard.
│   └─ test_engine.py 43 automated tests for engine.py
├─ backend/
│   └─ main.py        the web server: REST API, live updates, MQTT bridge
│   └─ warehouse.py   every action that changes the database
│                     (arrivals, reservations, confirmations, resets)
│   └─ plant.py        the pretend physical world (fake sensor readings)
│   └─ db.py           the database structure and starting data
│   └─ consistency.py  the read-only "is anything contradictory?" checker
│   └─ config.py       settings: network session name, physical constants
├─ dashboard/
│   └─ index.html      the webpage's structure (no text, no logic)
│   └─ labels.js        EVERY piece of on-screen text (FR/EN)
│   └─ app.js           the webpage's behaviour (buttons, live updates)
│   └─ twin.js          the live 3D view
├─ firmware/
│   └─ sketch.ino      the ESP32's own program
├─ tools/              the five test/verification scripts (section 12)
└─ docs/              detailed technical references and the demo script
```

14. Scoring — how this maps to the official brief

#	Requirement (CDC)	Points	Covered by this repository?
1	Identify the core / model	15	Yes — barcode + vision cross-check
2	Deduce the quantity	15	Yes — weight + vision cross-check
3	Track drying / 24h readiness	10	Yes — automatic, fixed 24h rule
4	Classify & locate by type/qty/date	15	Yes — dashboard stock & inventory panels
5	Automatically propose the right box	15	Yes — FIFO engine, partial picks, production batches
6	Mechanical design + animated 3D	25	No — separate CAD/mechanical deliverable, out of this repo's scope
7	Real-time interactive dashboard	15	Yes
8	Embedded simulation (ESP32)	15	Yes — Wokwi + Python fallback
9	Innovative idea	10	Yes — full FIFO audit trail + make-to-order production batches
10	Clear presentation, fluid demo	15	Prepared — see docs/demo-script.md

Numbering above follows the CDC's own criteria order as tracked in this project's README; row 6 (25 pts) is intentionally out of scope for this codebase and is being delivered as a separate CAD project.

15. Where things stand right now

Everything described in this document is implemented, tested, and committed to the project's version history as of "Contract 1.7":

- All identification, quantity, FIFO, quarantine, and drying rules above are live in the code and covered by passing automated tests.
- The dashboard reflects every one of these states in real time.
- The ESP32 firmware simulation runs the ready-to-use OLED display and reports raw sensor data honestly, with no shortcuts.
- A full manual code-review pass on this round of work additionally caught and fixed four smaller real bugs before they could surface during a live demo — including one where re-checking a quarantined box without explicitly stating a quantity could have wrongly re-quarantined a perfectly good box.

Bottom line. Every scored capability the CDC asks for — except the separate 25-point mechanical/CAD deliverable — has a working, tested, demonstrable answer in this repository today.

Smart Core Warehouse · National Robotics Weekend 8.0 · INSAT · Défi SOPAL & SOPALTEC

Document generated as a plain-language companion to the codebase — for the technical detail behind any statement here, see

[docs/contracts.md](#) and [README.md](#) in the project repository.